

TP — Créer un mode clair / mode sombre

Technologies : HTML / CSS / JavaScript

A. Objectifs

- créer une page web en HTML ;
- utiliser des variables CSS ;
- modifier l'apparence d'une page avec une classe CSS ;
- récupérer et modifier des éléments HTML avec JavaScript ;
- réagir à un clic ;
- utiliser `classList.toggle()` et `classList.contains()` ;
- modifier un texte avec `textContent` ;
- utiliser `localStorage` pour mémoriser un choix.

Le TP permet surtout de comprendre le rôle de chaque technologie :

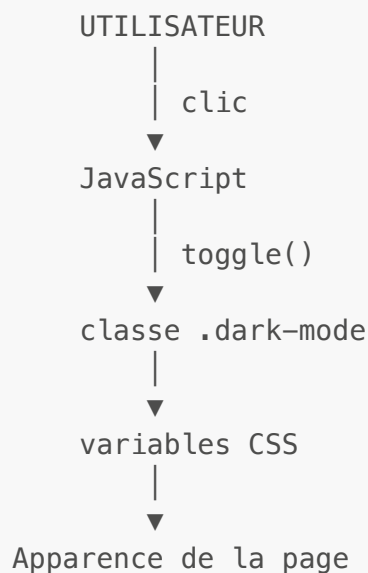
```
HTML      → structure
CSS        → apparence
JavaScript → comportement
```

B. Présentation du projet

Vous allez créer une page permettant de choisir entre deux thèmes :

- ☀ le mode clair ;
- 🌙 le mode sombre.

Le fonctionnement sera :



C. Organisation du projet

Créez un dossier :

```
tp-mode-sombre
```

avec les trois fichiers suivants :

```
tp-mode-sombre/  
├─ index.html  
├─ style.css  
└─ script.js
```

1. HTML — Créer l'interface

1.1. Structure de la page

Dans `index.html`, collez :

```
<!DOCTYPE html>  
<html lang="fr">  
  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  
  <title>Mode clair / mode sombre</title>  
  
  <link rel="stylesheet" href="style.css">  
</head>  
  
<body>  
  
  <main>  
  
    <h1>Bienvenue sur le site</h1>  
  
    <p>  
      Cette page peut être affichée en mode clair  
      ou en mode sombre.  
    </p>  
  
    <button id="themeButton">  
      🌙 Mode sombre  
    </button>
```

```
</main>

<script src="script.js"></script>

</body>

</html>
```

1.2. Questions

1. À quoi sert :

```
<link rel="stylesheet" href="style.css">
```

2. À quoi sert :

```
id="themeButton"
```

3. Pourquoi placer le `<script>` à la fin du `<body>` ?

2. CSS — Créer le mode clair

2.1. Créer les variables CSS

Dans `style.css` :

```
:root {
  --background: #ffffff;
  --text: #222222;
  --button: #222222;
  --button-text: #ffffff;
}
```

Une variable CSS possède un nom et une valeur :

```
--background: #ffffff;
```

Elle peut être utilisée avec :

```
var(--background)
```

2.2. Mettre en forme la page

Ajoutez :

```
body {  
  background-color: var(--background);  
  color: var(--text);  
  
  font-family: Arial, sans-serif;  
  text-align: center;  
  
  transition: 0.3s;  
}  
  
main {  
  max-width: 600px;  
  margin: 100px auto;  
}  
  
button {  
  background-color: var(--button);  
  color: var(--button-text);  
  
  border: none;  
  padding: 12px 20px;  
  border-radius: 8px;  
  
  cursor: pointer;  
}
```

Ouvrez dans le navigateur `index.html`.

Vous devez obtenir :

- un fond blanc ;
- du texte sombre ;
- un bouton sombre avec du texte blanc.

La page est maintenant en **mode clair**.

3. CSS — Créer le mode sombre

Ajoutez dans `style.css` :

```
.dark-mode {  
  --background: #121212;  
  --text: #eeeeee;  
  --button: #eeeeee;  
  --button-text: #121212;  
}
```

Pour l'instant, aucun changement n'est visible : le `<body>` ne possède pas encore la classe `dark-mode`.

On pourrait écrire :

```
<body class="dark-mode">
```

pour activer directement le mode sombre.

Objectif : JavaScript ajoutera ou supprimera cette classe automatiquement.

4. JavaScript — Récupérer le bouton

Dans `script.js` :

```
const button = document.getElementById("themeButton");
```

Cette instruction récupère l'élément HTML dont l'identifiant est `themeButton`. Il s'agit donc de notre bouton.

5. Détecter le clic

Ajoutez :

```
button.addEventListener("click", function () {  
  // Code exécuté lors du clic  
});
```

`addEventListener()` permet de réagir à un événement.

Ici :

```
"click"
```

signifie que le code sera exécuté lorsque l'utilisateur clique sur le bouton.

6. Changer de thème avec `toggle()`

Modifiez `script.js` :

```
const button = document.getElementById("themeButton");

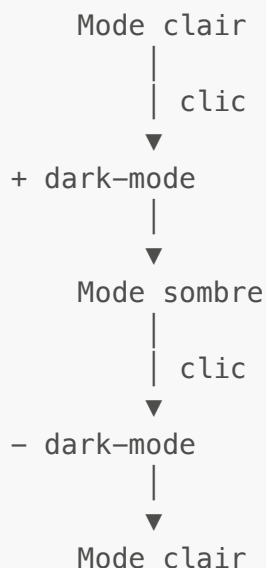
button.addEventListener("click", function () {

    document.body.classList.toggle("dark-mode");

});
```

Testez plusieurs clics.

`toggle()` ajoute la classe si elle est absente et la supprime si elle est présente.



Question

Que se passe-t-il si vous remplacez :

```
classList.toggle("dark-mode");
```

par :

```
classList.add("dark-mode");
```

Que se passe-t-il lorsque vous cliquez plusieurs fois ?

7. Modifier le texte du bouton

Le bouton affiche toujours :

 Mode sombre

Nous voulons afficher :

 Activer le mode sombre

en mode clair, puis :

 Activer le mode clair

en mode sombre.

7.1. Vérifier la présence de la classe

`classList.contains()` permet de vérifier si une classe est présente. La méthode renvoie `true` ou `false`.

```
document.body.classList.contains("dark-mode")
```

On peut donc écrire :

```
if (document.body.classList.contains("dark-mode")) {  
    // Mode sombre  
} else {  
    // Mode clair  
}
```

7.2. Modifier le texte

`button.textContent` permet de modifier le texte du bouton.

Le programme complet devient :

```
const button = document.getElementById("themeButton");

button.addEventListener("click", function () {

    document.body.classList.toggle("dark-mode");

    if (document.body.classList.contains("dark-mode")) {
        button.textContent = "☀️ Activer le mode clair";
    } else {
        button.textContent = "🌙 Activer le mode sombre";
    }

});
```

Testez votre programme.

8. Exercices

8.1. Ajouter une image

Ajoutez une image à votre page :

```

```

Votre page doit contenir :

- un titre ;
- une image ;
- un paragraphe ;
- un bouton.

Ajoutez :

```
img {
    max-width: 100%;
    border-radius: 10px;
}
```

8.2. Créer une carte

Ajoutez dans `index.html` :

```
<section class="card">

  <h2>Mon profil</h2>

  <p>
    Je programme actuellement en HTML, CSS et JavaScript.
  </p>

</section>
```

Puis dans `style.css` :

```
.card {
  margin-top: 30px;
  padding: 20px;

  background-color: #f2f2f2;

  border-radius: 10px;
}
```

Problème : La carte reste claire en mode sombre.

Objectif

Créez une variable `--card-background`.

Dans `:root` :

```
:root {
  --background: #ffffff;
  --text: #222222;
  --button: #222222;
  --button-text: #ffffff;
  --card-background: #f2f2f2;
}
```

Dans `.dark-mode` :

```
.dark-mode {
  --background: #121212;
  --text: #eeeeee;
  --button: #eeeeee;
  --button-text: #121212;
```

```
--card-background: #1e1e1e;  
}
```

Puis modifiez `.card` :

```
.card {  
  background-color: var(--card-background);  
}
```

La carte doit maintenant s'adapter au thème.

8.3. Personnaliser les couleurs

Essayez par exemple :

```
.dark-mode {  
  --background: #1e293b;  
  --text: #f8fafc;  
  --button: #f8fafc;  
  --button-text: #1e293b;  
  --card-background: #334155;  
}
```

Vous pouvez également modifier :

- la taille du titre ;
- la police ;
- les couleurs ;
- les bordures ;
- les arrondis ;
- les marges ;
- la taille de l'image.

8.4. Modifier la transition

Testez différentes valeurs :

```
transition: 0.1s;
```

```
transition: 0.5s;
```

```
transition: 1s;
```

Question : que se passe-t-il lorsque la durée augmente ?

9. Sauvegarder le thème

Pour le moment, le thème revient au mode clair après un rechargement de la page.

Nous allons utiliser `localStorage` pour mémoriser le choix de l'utilisateur.

9.1. Utiliser `localStorage`

`localStorage` permet de conserver des données dans le navigateur sous la forme :

```
clé → valeur
```

Pour enregistrer une donnée :

```
localStorage.setItem("theme", "dark");
```

Pour lire donnée :

```
localStorage.getItem("theme");
```

Par exemple :

```
const theme = localStorage.getItem("theme");

if (theme === "dark") {
  // appliquer le mode sombre
}
```

9.2. Compléter le programme

Le programme doit maintenant :

1. récupérer le thème sauvegardé au chargement ;
2. appliquer ce thème s'il existe ;
3. changer le thème lors d'un clic ;
4. sauvegarder le nouveau choix.

Le comportement attendu est :

```
Mode clair
  ↓ clic
Mode sombre
  ↓ rechargement
Mode sombre
```

et :

```
Mode sombre
  ↓ clic
Mode clair
  ↓ rechargement
Mode clair
```

Complétez votre programme en utilisant :

```
localStorage.setItem("theme", "dark");
localStorage.setItem("theme", "light");
```

et :

```
const savedTheme = localStorage.getItem("theme");
```

Le comportement attendu est :

```
Mode clair → clic → mode sombre → rechargement → mode sombre
```

et :

```
Mode sombre → clic → mode clair → rechargement → mode clair
```

Correction possible

Après avoir recherché votre propre solution, comparez-la avec :

```
const button = document.getElementById("themeButton");
```

```
// Récupérer le thème sauvegardé
const savedTheme = localStorage.getItem("theme");

// Appliquer le thème sauvegardé
if (savedTheme === "dark") {
    document.body.classList.add("dark-mode");
    button.textContent = "☀️ Activer le mode clair";
} else {
    button.textContent = "🌙 Activer le mode sombre";
}

// Gérer le clic
button.addEventListener("click", function () {

    document.body.classList.toggle("dark-mode");

    if (document.body.classList.contains("dark-mode")) {
        button.textContent = "☀️ Activer le mode clair";
        localStorage.setItem("theme", "dark");
    } else {
        button.textContent = "🌙 Activer le mode sombre";
        localStorage.setItem("theme", "light");
    }

});
```

Le programme réalise maintenant trois actions :

- modifier l'apparence ;
- modifier le texte du bouton ;
- mémoriser le choix de l'utilisateur.

10. TP final

Ajoutez au moins **deux** améliorations parmi les suivantes :

- ☐ ajouter une animation ;
- ☐ modifier l'icône du bouton ;
- ☐ détecter automatiquement le thème du système ;
- ☐ ajouter un bouton pour réinitialiser le thème.

10.1. Critères de réussite

Critère	Réussi
La page HTML fonctionne	☐
Le CSS est chargé	☐
Le mode clair fonctionne	☐

Critère	Réussi
Le mode sombre fonctionne	☐
Le bouton réagit au clic	☐
<code>classList.toggle()</code> est utilisé	☐
Le texte du bouton change	☐
Les variables CSS sont utilisées	☐
La carte ou l'image s'intègre correctement	☐
Le code est correctement indenté	☐
Deux fonctionnalités supplémentaires sont ajoutées	☐

10.2. Exemple de corrigé — Détecter le thème du système

JavaScript permet de tester une préférence de thème du système avec :

```
window.matchMedia("(prefers-color-scheme: dark)")
```

Pour savoir si le thème sombre est actif :

```
window.matchMedia("(prefers-color-scheme: dark)").matches
```

Cette expression renvoie `true` ou `false`.

Objectif : si aucun thème n'est sauvegardé, utiliser la préférence du système.

11. Questions

1. Quel langage permet de créer la structure de la page ?
2. Quel langage permet de gérer les couleurs et la mise en forme ?
3. Quel langage permet de réagir au clic ?
4. À quoi servent les variables CSS ?
5. À quoi sert la classe `.dark-mode` ?
6. À quoi sert `document.getElementById("themeButton")` ?
7. À quoi sert `addEventListener("click", ...)` ?
8. Que fait `classList.toggle("dark-mode")` ?
9. À quoi sert `classList.contains("dark-mode")` ?
10. À quoi sert `button.textContent` ?
11. À quoi sert `localStorage` ?

Réponses :

1.
2.
3.
4.
5.
6.
7.
8.
9.
10.
11.

12. Tableau récapitulatif des notions

Technologie	Notions principales	Rôle
HTML	<code><link></code> , <code>id</code> , <code><button></code>	Structure de la page
CSS	variables, classes, <code>var()</code> , <code>transition</code>	Apparence
JavaScript	<code>getElementById()</code> , <code>addEventListener()</code>	Interaction
JavaScript	<code>classList.toggle()</code>	Ajouter/supprimer une classe
JavaScript	<code>classList.contains()</code>	Vérifier une classe
JavaScript	<code>textContent</code>	Modifier un texte
localStorage	<code>setItem()</code> , <code>getItem()</code>	Mémoriser le thème
JavaScript	<code>matchMedia()</code>	Détecter une préférence du système

Fonctionnement global

```

Utilisateur
  ↓ clic
JavaScript
  ↓
toggle("dark-mode")
  ↓
Ajout / suppression de .dark-mode
  ↓
Variables CSS
  ↓
Nouvelle apparence
  ↓
Mise à jour du bouton
  ↓
localStorage

```

Au rechargement :

```
Chargement
↓
Lecture de localStorage
↓
Récupération du thème
↓
Application du thème
```

Conclusion

HTML	→ structure
CSS	→ apparence
JavaScript	→ comportement
localStorage	→ mémorisation

L'objectif du TP est de comprendre comment ces technologies peuvent travailler ensemble pour créer une interface interactive.
